

AUDITORIA FORENSE FLUXYRA — COMPLEMENTAÇÃO DA ETAPA 1

Execução real com instrumentação, testes seguros e evidências cruas da PiAPI

Data da execução: 2026-07-26 (04:15Z a 04:30Z, UTC)

Ambiente: produção local — `fluxyra.service` (porta 3000) → `https://5be3213ef.abacusai.cloud`

Branch: `feat/f3-piapi-landing-pricing` | **Commits desta etapa:** `9aba8c8` (instrumentação)

Usuário de teste: `robimlocar@gmail.com` (`f7262383-9233-46b6-ae2e-5f8403208a52` , plano `agency`)

GARANTIAS CUMPRIDAS:

- ✓ **NENHUMA task nova foi criada na PiAPI** — zero POSTs à PiAPI; apenas 3 GETs read-only em `/task/{id}` de tasks já existentes.
 - ✓ **Nenhum crédito foi consumido** — saldo era 539 antes e é 539 depois (alterado temporariamente para 1 no teste 5 e **restaurado imediatamente**, com confirmação via `/api/me`).
 - ✓ Serviço `fluxyra.service` terminou **ativo e saudável** (HTTP 200).
 - ✓ Todos os secrets estão **mascarados** (API key PiAPI: `e63a****c959` ; tokens/cookies: `<MASCARADO>`).
-

1. RESUMO DO QUE FOI EXECUTADO (comandos + horários UTC)

Hora (UTC)	Ação	Resultado
~04:10	Instrumentação de 5 arquivos (ver §2)	código alterado
04:15	<code>npx tsc --noEmit</code>	rc=0 (sem erros de tipo)
04:17	<code>npm run build</code>	rc=0 (build OK)
04:19:40	<code>sudo systemctl restart fluxyra.service</code>	active ; curl localhost:3000/ → HTTP 200 em 0,20s
04:19:52	<code>git commit</code> → 9aba8c8 (5 arquivos, +414/−37)	instrumentação versionada
04:20	Geração de sessão de teste (admin generate_link + verify , service role)	cookie SSR válido; /api/me → {"credits": 539, "plan": "agency"}
04:22:37	Teste 1 — POST /api/generate/video sem auth	HTTP 401 em 0,012s
04:22:47	Testes 2a/2b — prompt vazio / sem model_uuid	HTTP 400 em 0,094s / 0,076s
04:22:47	Teste 3 — model_uuid inexistente	HTTP 404 em 0,173s
04:23:12	Teste 5 — saldo reduzido 539→1 (UPDATE via service role)	http 200 novo saldo: 1
04:23:13	Teste 5 — POST com Seedance 2.0 (68 cr)	HTTP 402 em 0,150s
04:23:13	Teste 5 — saldo RESTAURADO 1→539 + confirmação /api/me	{"credits": 539, "plan": "agency"}
04:23:4x	SELECT generations com status=failed (Supabase REST, service role)	7 registros
04:23:57-58	3× GET read-only https://api.piapi.ai/api/v1/task/{id}	HTTP 200 (0,55s / 0,25s / 0,25s)

Testes 4 (aspect ratio inválido) e 6 (image_url malformada) NÃO foram executados — o código **não possui validação local** desses campos; a requisição iria direto para a PiAPI e criaria uma task real, o que é proibido nesta etapa. Documentados como achado em §5 e §7.

2. INSTRUMENTAÇÃO IMPLEMENTADA (commit `9aba8c8`)

Logging estruturado JSON com `requestId` (uuid de 8 chars), timestamp ISO, escopo, evento, duração em ms e dados relevantes. Secrets mascarados por `maskSecret()` (4 primeiros + 4 últimos chars).

Arquivo	O que foi instrumentado
src/lib/audit-log.ts (novo)	auditLog(scope, event, requestId, data?, ms?) → imprime [FLUXYRA-AUDIT] {json} no stdout (vai para logs/app.log via systemd); newRequestId(), maskSecret(), truncate()
src/lib/piapi/client.ts	piapiFetch (l.75): loga request (método/URL/headers com x-api-key mascarada /body truncado) e response (HTTP status + body completo incl. logs[] , l.115); buildVideoPayload : loga entrada/payload final; submitVideoTask (l.614): entrada/sucesso(task_id)/falha + ms; getTaskStatus (l.737): status, logs[] da PiAPI (l.753), error + ms
src/app/api/generate/video/route.ts	eventos por etapa: auth_falhou_401 (l.19), entrada (body completo), validacao_falhou_400 (l.72), modelo_nao_encontrado_404 (l.93), creditos_insuficientes_402 (l.122, com saldo/custo/modelo), plano_insuficiente_403, creditos_debitados (l.154, antes/custo/depois), generation_criada, sucesso_200, piapi_erro_estorno, execucao_500
src/app/api/generate/status/route.ts	entrada, task_failed_piapi (l.224, com provider_error + piapi_logs completos , l.228), download_concluido (bytes/ms), upload_storage_ok/falhou, storage_excecao_fallback_provider, falha_persistida_estorno, ainda_processando, execucao_500
src/components/studio/generation-dock.tsx	genLog() (l.235) → [FLUXYRA-GEN] {json} no console do navegador; handleGenerate.entrada (l.1099, estado completo: modelo/aspect/duração/créditos/refs), handleGenerate.bloqueado (motivos: prompt_vazio/modelo_indisponivel/creditos_insuficientes, l.1121-1133), submit.request (l.1045, body completo), submit.response (l.1070, HTTP status + resposta), submit.failed / submit.success

Exemplo REAL de log gerado (extraído de logs/app.log durante o teste 5):

```
[FLUXYRA-AUDIT]
{"ts":"2026-07-26T04:23:13.075Z","scope":"api.generate.video","event":"entrada","requestId":"e9340b8d","data":{"user_id":"f7262383-9233-46b6-ae2e-5f8403208a52","body":{"prompt":"teste auditoria saldo insuficiente","model_uuid":"93c9f7db-d47d-4824-9214-aea12b7edcb1"}}}
[FLUXYRA-AUDIT]
{"ts":"2026-07-26T04:23:13.140Z","scope":"api.generate.video","event":"creditos_insuficientes_402","requestId":"e9340b8d","ms":96,"data":{"credits_balance":1,"credit_cost":68,"model":"Seedance 2.0"}}}
```

Com o `requestId` é possível correlacionar clique na UI (`[FLUXYRA-GEN]` no console do navegador) → rota API → chamada PiAPI → resposta, com o tempo de cada etapa.

Observação: os eventos `[FLUXYRA-GEN]` do frontend aparecem no **console do navegador** (F12), pois rodam no cliente. Nesta etapa não houve interação via UI (testes foram via curl), então os exemplos reais capturados são todos `[FLUXYRA-AUDIT]` do backend.

3. EVIDÊNCIAS DOS TESTES REAIS (Parte B)

Todos via `curl` contra `http://localhost:3000` (o mesmo processo que serve a URL pública). Cookie de sessão: `sb-pckfdyrhksdkwakptyuk-auth-token=<MASCARADO>`.

Teste 1 — Sem autenticação → 401

```
POST /api/generate/video (sem cookie)
Body: {"prompt":"teste auditoria sem auth","model_uuid":"5a1d0c40-025e-4c81-a8e9-5472200a2764"}
→ HTTP 401 | 0,012s | {"error":"Não autorizado"}
```

Log: `{"event":"auth_falhou_401","requestId":"8e1440ac","ms":1}` — falha na l.19 de `video/route.ts`, **antes** de qualquer débito/PiAPI.

Teste 2 — Prompt vazio / model_uuid ausente → 400

```
2a: {"prompt":"","model_uuid":"5a1d0c40-..."} → HTTP 400 | 0,094s | {"error":"Prompt e modelo são obrigatórios"}
2b: {"prompt":"teste auditoria"} → HTTP 400 | 0,076s | {"error":"Prompt e modelo são obrigatórios"}
```

Logs: `{"event":"validacao_falhou_400","requestId":"2ec22738","ms":42,"data":{"has_prompt":false,"has_model_uuid":true}}` e `{"requestId":"0330f6ca","ms":26,"data":{"has_prompt":true,"has_model_uuid":false}}` — l.70-80 de `video/route.ts`.

Teste 3 — Modelo inexistente → 404

```
{"prompt":"teste auditoria modelo inexistente","model_uuid":"00000000-0000-0000-0000-000000000000"}
→ HTTP 404 | 0,173s | {"error":"Modelo não encontrado"}
```

Log: `{"event":"modelo_nao_encontrado_404","requestId":"e4bcd0ce","ms":121}` — l.93.

Teste 4 — Aspect ratio inválido → NÃO EXECUTADO ⚠ (achado)

`video/route.ts` valida apenas `prompt` e `model_uuid` (l.70). **Não existe validação local de `aspect_ratio`** — um valor inválido (ex.: `"7:3"`) passaria por `buildVideoPayload` e chegaria à PiAPI, criando uma task real (proibido nesta etapa). O comportamento (rejeição, fallback ou cobrança) só pode ser comprovado com geração manual — ver §7.

Teste 5 — Saldo insuficiente → 402, sem débito, sem PiAPI ✓

Saldo real lido imediatamente antes: **539**. `UPDATE profiles SET credits_balance=1` (service role) às 04:23:12Z.

```
{ "prompt": "teste auditoria saldo insuficiente", "model_uuid": "93c9f7db-..." } (Seedance 2.0, 68 cr)
→ HTTP 402 | 0,150s | { "error": "Créditos insuficientes", "required": 68, "available": 1 }
```

Log: `{ "event": "creditos_insuficientes_402", "requestId": "e9340b8d", "ms": 96, "data": { "credits_balance": 1, "credit_cost": 68, "model": "Seedance 2.0" } }`.

Nenhum evento `piapi.*` foi gerado para o requestId `e9340b8d` (confirmado por `grep`) — a checagem (l.~120) ocorre **antes** do débito (l.154) e da PiAPI. Saldo **restaurado para 539** às 04:23:13Z; confirmação: `/api/me` → `{ "credits": 539, "plan": "agency" }`.

Teste 6 — image_url malformada → NÃO EXECUTADO ⚠ (achado)

Igual ao teste 4: **não há validação local** de `start_image_url` / `end_image_url` / `reference_images` (formato, domínio, acessibilidade). Uma URL malformada chegaria à PiAPI dentro de `image_urls` e criaria task real. Requer geração manual — ver §7.

4. RESPOSTAS CRUAS DA PiAPI (Parte C — somente GET read-only)

4.1 Generations com `status=failed` no banco (SELECT via service role, 04:23Z)

created_at (UTC)	Modelo	provider_task_id	error_message no banco
2026-07-26 03:51:48	Seedance 2.0	59d36e05-29ea-...	“Imagem rejeitada: a foto contém rosto de pessoa real...” (já em PT-BR ✓)
2026-07-24 03:15:51	Flux Schnell	—	(nulo)
2026-07-24 03:14:44	Flux Schnell	—	(nulo)
2026-07-24 03:12:09	Flux Schnell	—	PiAPIError: PiAPI error 400
2026-07-24 03:10:02	Flux Schnell	—	PiAPIError: PiAPI error 400
2026-07-24 02:59:05	Flux Schnell	—	PiAPIError: PiAPI error 401
2026-07-23 17:11:53	Flux Schnell	b17ae5d3-d6d8-...	(nulo)

A generation histórica `eb7e5baa-...` (task `41bd6e24-...`, o caso “Internal Error” original) **não existe mais no banco** — foi excluída (provavelmente apagada pelo usuário na UI). A task, porém, continua consultável na PiAPI (abaixo).

4.2 GET `https://api.piapi.ai/api/v1/task/{id}` — header `x-api-key: e63a****c959`

(a) Task `41bd6e24-a2fe-4bee-a4da-cf2d0557cb1c` (caso original “Internal Error”) — HTTP 200 em 0,55s às 04:23:57Z:

```

"model": "seedance", "task_type": "seedance-2", "status": "failed",
"error": {"code": 10000, "raw_message": "", "message": "Internal error. Please try again later.", "detail": null},
"logs": [
  "mode auto-detected: first_last_frames (images=1, videos=0, audios=0)",
  "audio: enabled=true",
  "The request was rejected because the input image may contain a real person. For -less-restriction task types, you can set `auto_upload_assets: true` in your request to have the image uploaded as a virtual asset before generation. You may also manually upload and manage virtual assets via the asset API (GET/POST/DELETE /api/v1/asset/...) to control asset creation. Otherwise please use a different image.",
  "Attempt 1 failed (content restriction), retrying.",
  "Internal error. Please try again later.",
  "restored frozen points",
  "Internal error. Please try again later."
],
"meta": {"created_at": "2026-07-26T02:37:28Z", "started_at": "02:37:31Z", "ended_at": "02:37:45Z"}

```

(b) Task 59d36e05-29ea-412b-b798-b2065fedec40 (falha de 03:51, PÓS-commit a93267b) — HTTP 200 em 0,25s: **status, error e logs IDÊNTICOS** ao caso (a) — mesma rejeição “input image may contain a real person”. Input ecoado pela PiAPI: `mode=first_last_frames`, `resolution=1080p`, `aspect_ratio=9:16`, 1 imagem em `image_urls` (Supabase Storage).

(c) Task b17ae5d3-d6d8-47ff-b92f-442385314294 (Flux Schnell, 23/07) — HTTP 200 em 0,25s: **status completed** na PiAPI (error code 0, sem logs), porém marcada como **failed no banco** com `error_message` nulo → falha ocorreu na fase de **polling/persistência do lado Fluxyra**, não na PiAPI.

4.3 Cadeia de evidências do caso “Internal Error” / “Imagem rejeitada”

- Banco (antes):** generation eb7e5baa-... gravada com `error_message = "Internal error. Please try again later."` (mensagem genérica, hoje excluída do banco).
 - PiAPI (resposta crua, §4.2a):** `error.message` é genérico, mas a causa real está em `logs[2]`: “input image may contain a real person” → **content restriction / anti-deepfake** do Seedance.
 - Código ANTES do commit a93267b :** `getTaskStatus` não retornava `logs[]`, e `status/route.ts` persistia apenas `error.message` → o usuário via só “Internal Error”.
 - Código DEPOIS (a93267b , mantido em 9aba8c8):** `getTaskStatus` retorna `logs[]` (`client.ts` l.753); `status/route.ts` l.230-250 concatena os logs e traduz: contém “real person”/“content restriction” → “Imagem rejeitada: a foto contém rosto de pessoa real...” (l.238); “plan limit”/“active task count” → mensagem de limite; “nsfw”/“content policy” → mensagem de política.
- Comprovação de funcionamento em produção:** a falha de 03:51 (task 59d36e05) já foi persistida no banco com a mensagem PT-BR correta (§4.1, linha 1).

⚠ ACHADO NOVO E COMPROVADO — o workaround auto_upload_assets NÃO resolve a rejeição: o `logs/app.log` registra que o payload enviado às 03:51 (e em pelo menos 4 outras tentativas: tasks c6c8e10f , fcb61799 , 44cf6344 , 3f4de519) **já continha "auto_upload_assets": true** — e a PiAPI rejeitou mesmo assim, com a mesma mensagem. Trecho real do log:

```

[video/generate] PAYLOAD {"model":"seedance","task_type":"seedance-2","input":
{...,"image_urls":[...]},"config":{"service_mode":"public"},"auto_upload_assets:true}
[status] FAILED task_id= 59d36e05-... provider_msg= Internal error. ... logs= [..., "The
request was rejected because the input image may contain a real person. For -less-re-
striction task types, you can set `auto_upload_assets: true` ...", ...]

```


A dica da PiAPI refere-se a task types `-less-restriction` — o task_type usado é `seedance-2` (padrão), para o qual o flag não contorna o bloqueio anti-deepfake. Conclusão: **imagens com rosto de pessoa real são incompatíveis com `seedance-2`**; as opções são usar imagem sem rosto real, outro modelo (ex.: Kling), ou avaliar a existência de um task type “less restriction” na PiAPI.

4.4 Erros históricos de SUBMISSÃO capturados no app.log (conta PiAPI)

Dois erros adicionais registrados no log do serviço, ocorridos na **submissão** (POST da task, antes de existir task_id):

- `PiAPIError: active task count 2 has reached the plan limit` — o plano PiAPI atual permite só **2 tasks Seedance simultâneas**; a 3ª submissão é recusada.
 - `PiAPIError: insufficient credits` — **créditos da conta PiAPI** (do dono do SaaS, não do usuário Fluxyra) esgotados no momento.
-

5. LOCALIZAÇÃO EXATA DA FALHA POR TIPO DE ERRO

Erro observado	Etapas do fluxo	Arquivo / linha (estado atual)
401 Não autorizado	Backend, antes de tudo	<code>src/app/api/generate/video/route.ts</code> l.14-24
400 Prompt/modelo obrigatórios	Backend, validação de entrada	idem, l.70-80
404 Modelo não encontrado	Backend, lookup em <code>ai_models</code>	idem, l.84-99
402 Créditos insuficientes	Backend, checagem de saldo (antes do débito)	idem, l.~118-131
“Internal Error” genérico (Seedance rosto real)	PiAPI, fase de processamento (task criada, falha ~14s depois)	causa real só em <code>logs[]</code> ; tradução em <code>src/app/api/generate/status/route.ts</code> l. 230-250
“plan limit” (2 tasks simultâneas)	PiAPI, fase de submissão (HTTP de erro no POST)	lançado por <code>piapiFetch / submitVideoTask (src/lib/piapi/client.ts</code> l.75/614); estorno em <code>video/route.ts</code> (bloco catch, evento <code>piapi_erro_estorno</code>)
“insufficient credits” (conta PiAPI)	PiAPI, fase de submissão	idem acima
Flux PiAPI error 400/401 (24/07)	PiAPI, submissão (rota de imagem)	mensagens genéricas persistidas sem detalhe do body — mesma classe de problema corrigida para vídeo em <code>a93267b</code>
Task <code>b17ae5d3</code> completed na PiAPI mas failed no banco	Polling/persistência no Fluxyra (pós-PiAPI)	fluxo de status de imagem; <code>error_message</code> nulo impede diagnóstico — agora a instrumentação de <code>status/route.ts</code> capturaria <code>(url_nao_extraida , upload_storage_falhou , excecao_500)</code>
<code>aspect_ratio</code> inválido / <code>image_url</code> malformada	Sem validação local — iria à PiAPI	<code>video/route.ts</code> valida só <code>prompt/model</code> (l.70); nenhuma checagem de <code>aspect_ratio</code> ,

Erro observado	Etapa do fluxo	Arquivo / linha (estado atual)
		start_image_url , reference_images

6. TABELA DE CAUSA RAIZ (somente causas COMPROVADAS)

Erro	Causa raiz com-provada	Evidência	Arquivo	Linha	Correção
“Internal Error. Please try again later.” (Seedance c/ imagem)	Bloqueio anti-deepfake da PiAPI: imagem de referência contém rosto de pessoa real; <code>error.message</code> da PiAPI é genérico e a causa só vem em <code>logs[]</code>	GET tasks 41bd6e24 e 59d36e05 (\$4.2): <code>logs[2] = “input image may contain a real person”;</code> código 10000	<code>src/app/api/generate/status/route.ts</code>	230-250	Já aplicada (a93267b): parsing de <code>logs[]</code> + mensagem PT-BR. Comprovadamente funcionando (falha de 03:51 persistida traduzida). Pendente: bloquear/alertar upload de imagem com rosto p/ Seedance (detecção prévia) ou oferecer modelo alternativo
Mesmo erro persiste APÓS <code>auto_upload_assets:true</code>	O flag só vale para task types “-less-restriction”; em <code>seedance-2</code> não contorna o bloqueio	<code>logs/app.log</code> : payloads com <code>"auto_upload_assets":true</code> (tasks 59d36e05 , fcb61799 , 44cf6344 , 3f4de519) seguidos de FAILED com a mesma mensagem	<code>src/lib/piapi/client.ts</code>	422	Remover a expectativa de que o flag resolve; tratar rosto real como incompatível com <code>seedance-2</code> (validação prévia ou troca de modelo)

Erro	Causa raiz com-provada	Evidência	Arquivo	Linha	Correção
“active task count 2 has reached the plan limit”	Limite do plano da conta PiAPI: máx. 2 tasks Seedance simultâneas	logs/app.log l.64-66: PiAPIError: active task count 2 has reached the plan limit na submissão	src/lib/piapi/client.ts (origem) / status.route (tradução PT-BR já existe)	75/614	Fila/limite de concorrência no backend por provedor; upgrade do plano PiAPI
“insufficient credits” (submissão)	Créditos da conta PiAPI esgotados (independe do saldo do usuário Fluxyra)	logs/app.log l.134-136: PiAPIError: insufficient credits	src/lib/piapi/client.ts	75/614	Monitorar saldo PiAPI; mensagem específica ao usuário (“instabilidade do provedor”) + alerta ao admin
402 sem débito indevido	Checação de saldo ocorre antes do débito e da PiAPI	Teste 5 (§3): 402 em 0,15s, zero eventos piapi.*, saldo intacto	src/app/api/generate/video/route.ts	~118-131	Nenhuma (comportamento correto confirmado)
Ausência de validação local de aspect_ratio/image_url	Rota valida apenas prompt e model_uuid	Leitura do código (l.70) + testes 4/6 não executáveis com segurança	src/app/api/generate/video/route.ts	70	Adicionar validação de aspect_ratio contra lista suportada por modelo e validação de URL (formato/domínio) antes do débito

PENDENTE DE REPRODUÇÃO MANUAL (hipóteses NÃO comprovadas nesta etapa)

- **Task b17ae5d3 (Flux) completed na PiAPI mas failed no banco:** hipóteses — exceção no download/upload para o Storage, ou polling interrompido. `error_message` nulo impede confirmação; a instrumentação atual capturaria a etapa exata numa recorrência.
- **Flux PiAPI error 400/401 (24/07):** 401 sugere API key inválida/trocada naquele momento e 400 payload inválido, mas o corpo da resposta não foi persistido na época — não comprovável retroativamente.
- **Comportamento com aspect_ratio inválido e image_url malformada:** exige criar task real (proibido nesta etapa).
- **Timeout de polling e falha de upload no Storage em vídeo:** sem ocorrência registrada nos dados disponíveis; requer reprodução ao vivo.

7. TESTES QUE EXIGEM GERAÇÃO MANUAL (instruções para o usuário)

A instrumentação (9aba8c8) está ativa em produção. Em qualquer geração feita agora, **tudo será capturado automaticamente**: no servidor, `logs/app.log` registra cada etapa com `[FLUXYRA-AUDIT]` (request completo, payload PiAPI, resposta crua com `logs[]`, tempos, estornos); no navegador, o console (F12) registra `[FLUXYRA-GEN]` (estado do dock, request/response do submit). Basta gerar e me informar o horário — eu extraio as evidências pelo `requestId`.

7.1 Reproduzir o “Internal Error”/imagem rejeitada ao vivo (Seedance):

1. Acesse <https://5be3213ef.abacusai.cloud/studio>, aba Vídeo, modelo **Seedance 2.0**.
2. Anexe uma **foto com rosto de pessoa real** como imagem de referência.
3. Prompt curto qualquer (ex.: “pessoa apresentando um produto, 5s”) e clique em Gerar.
4. Resultado esperado: falha em ~10–20s com a mensagem PT-BR “Imagem rejeitada: a foto contém rosto de pessoa real...” e **estorno automático** dos 68 créditos (confira o saldo antes/depois).

7.2 Confirmar que Seedance funciona sem rosto real: repita 7.1 com imagem **sem pessoas** (produto/paisagem). Esperado: geração conclui normalmente.

7.3 Testar aspect_ratio inválido: não é possível pela UI (o seletor só oferece valores válidos); seria via curl com task real. Recomendo **implementar a validação local** (§6) em vez de testar contra a PiAPI.

7.4 Reproduzir limite de concorrência (plan limit): dispare **3 gerações Seedance em sequência rápida** (menos de ~30s entre elas). Esperado: a 3ª falha imediatamente na submissão com estorno; log registrará `active task count 2 has reached the plan limit`.

7.5 Timeout/falha de storage: sem passo determinístico para forçar; se ocorrer em uso normal, os eventos `download_concluido`, `upload_storage_falhou`, `storage_excecao_fallback_provider` e `excecao_500` no `app.log` identificarão a etapa exata.

Estado final do ambiente

- `fluxyra.service` : **active** (restart 04:19:40Z; `curl /` → HTTP 200).

- Saldo do usuário: **539 créditos** (idêntico ao início; alteração temporária do teste 5 revertida e confirmada).
- **Zero tasks criadas na PiAPI** nesta etapa (apenas 3 GETs read-only).
- Commits: 9aba8c8 (instrumentação) + commit deste relatório.